SmartPDF Chatbot)

Kartik Tayal

1 Introduction

This project implements a Retrieval-Augmented Generation (RAG) based AI assistant capable of answering questions from PDF documents using semantic retrieval and Large Language Models (LLMs).

The system combines document processing, embedding generation, vector similarity search, and response generation to provide context-aware answers grounded in uploaded documents.

2 Project Objective

The objective of this project is to build an intelligent document question-answering system capable of:

- Reading and processing PDF documents
- Understanding semantic meaning from textual data
- Retrieving contextually relevant information
- Generating grounded and accurate responses

3 Technology Stack

- Python
- LangChain
- ChromaDB
- HuggingFace Embeddings
- Google Gemini API
- PyPDF

4 System Workflow

The complete RAG workflow used in this project is shown below:

PDF → Text Extraction → Chunking → Embeddings → Vector Database → Semantic Retrieval → Ge

5 RAG Pipeline Flowchart

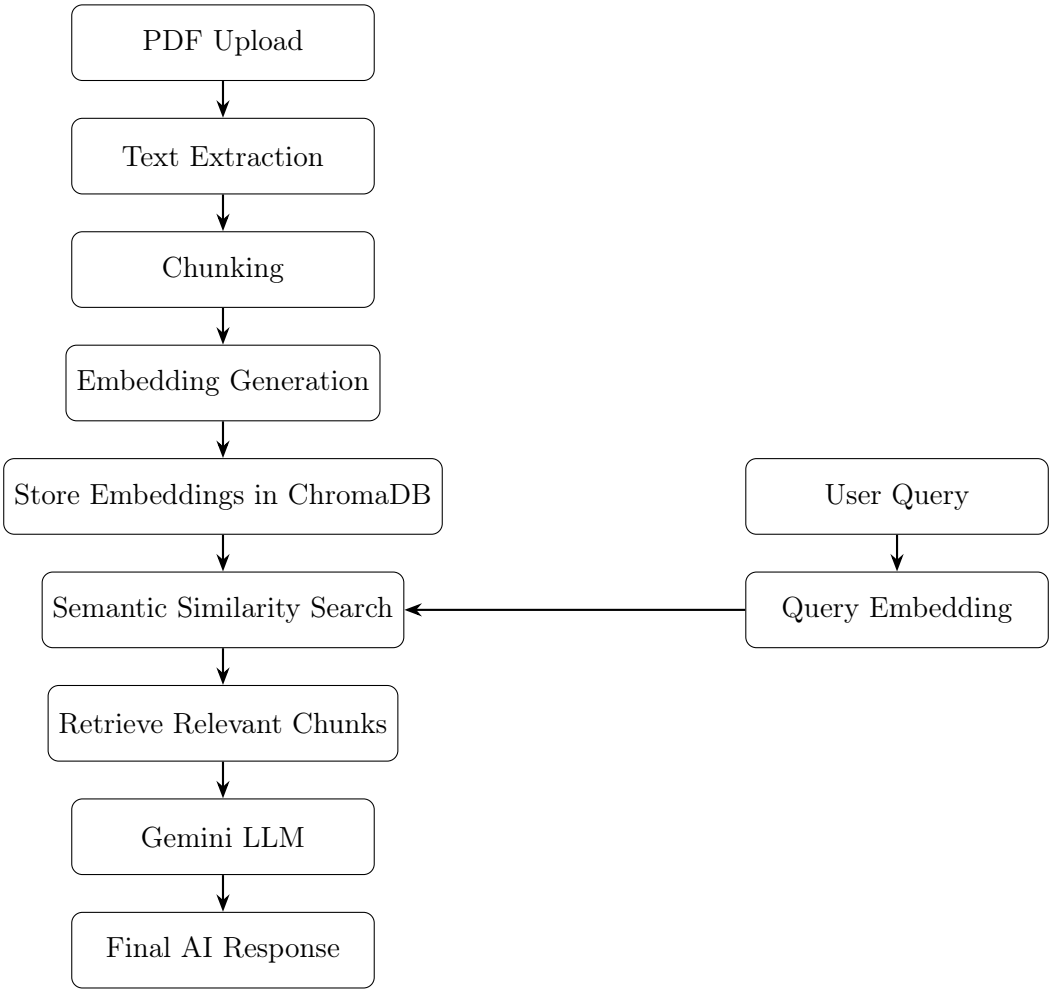

Figure 1: Retrieval-Augmented Generation (RAG) Pipeline

6 Implementation Details

6.1 PDF Loading

PDF documents are loaded using `PyPDFLoader` from LangChain.

6.2 Text Chunking

The extracted text is divided into smaller overlapping chunks using `RecursiveCharacterTextSplitter`. This improves retrieval quality and contextual continuity.

6.3 Embedding Generation

Each chunk is converted into dense vector embeddings using the HuggingFace embedding model:

`sentence-transformers/all-MiniLM-L6-v2`

6.4 Vector Database Storage

The generated embeddings are stored inside ChromaDB, enabling efficient semantic similarity search.

6.5 Semantic Retrieval

When the user asks a question, the query is converted into embeddings and matched against stored vectors using cosine similarity.

6.6 Response Generation

The retrieved chunks are sent to Gemini LLM, which generates grounded and context-aware responses.

7 Challenges Faced

- Gemini SDK integration issues
- Environment configuration problems
- Processing large PDF documents
- Improving retrieval quality
- Handling noisy PDF text extraction

8 Key Learnings

- Understanding Retrieval-Augmented Generation
- Semantic similarity search
- Importance of chunking strategies
- Vector database implementation
- Hallucination reduction techniques

9 Future Improvements

- Multi-PDF support
- Streamlit web interface
- Conversational memory
- Hybrid search implementation
- Source citation support

- OCR support for scanned PDFs
- Multimodal document understanding

10 Conclusion

This project demonstrates how Retrieval-Augmented Generation (RAG) improves the reliability and contextual accuracy of Large Language Models by grounding responses using retrieved document context.